

6.9 PENENTUAN INDEKS

Pada kebanyakan DBMS, apabila suatu relasi memiliki kunci primer, kunci tersebut dengan sendirinya akan bertindak sebagai indeks. Itulah sebabnya, jika Anda bermaksud mencari data menurut kunci primer, pencarian data akan berjalan dengan cepat.

Hal yang menjadi masalah adalah bahwa setiap relasi hanya memungkinkan satu kunci primer. Lalu, bagaimana kalau dikehendaki untuk mengambil data yang tidak didasarkan pada kunci primer? Tentu saja, Anda tetap bisa melakukannya walau tanpa indeks. Namun, kendalanya terletak pada waktu tanggapannya. Data yang dicari akan ditemukan dengan waktu yang agak lama, bergantung pada letak data dalam relasi dan jumlah record dalam relasi.

Kalau pencarian data sering dilakukan, Anda perlu menciptakan indeks terhadap data yang menjadi kunci pencarian tersebut. Namun, hal penting yang perlu diketahui, menciptakan indeks untuk semua kolom dalam relasi adalah hal yang berlebihan dan bukan ide yang baik. Mengapa begitu? Tak lain adalah karena Anda harus membayar mahal atas tindakan tersebut. Lalu, apa yang dibayar mahal? Jawabannya adalah kinerja untuk memutakhirkan indeks ketika terdapat operasi penambahan, pengubahan, atau penghapusan data. Selain itu, diperlukan ruang dalam penyimpanan eksternal untuk mencatat indeks. Oleh karena sifatnya yang demikian, indeks hanya perlu dilakukan pada kolom-kolom yang memang memerlukannya.

Panduan untuk menentukan indeks adalah seperti berikut:

1. Kunci primer perlu diindeks secara eksplisit atau tidak bergantung pada DBMS yang digunakan. DBMS modern secara otomatis mengindeks kolom-kolom yang dijadikan sebagai kunci primer.
2. Indeks sangat bermanfaat untuk relasi yang berukuran besar (jumlah baris ribuan atau jutaan).
3. Indeksilah kolom-kolom selain kunci primer yang sering menjadi kunci pencarian. Kolom-kolom yang dimaksud bisa saja tidak berkedudukan sebagai kunci asing. Sebagai pegangan, kolom-kolom yang perlu diindeks adalah yang sering muncul dalam klausa WHERE. Kunci asing perlu diindeks karena sering muncul dalam klausa WHERE ketika terjadi operasi penggabungan beberapa relasi.
4. Indeksilah kolom-kolom yang sering dilibatkan dalam klausa ORDER BY atau GROUP BY dalam perintah SQL dan perintah SQL tersebut sering kali dieksekusi.

Perintah `CREATE INDEX` berguna untuk membuat indeks tidak-unik, yakni indeks yang memungkinkan nilai kolom sejumlah *record* boleh sama. Bentuk penggunaan perintah ini serupa dengan `CREATE UNIQUE INDEX`. Contoh:

```
CREATE INDEX indeks_kota  
ON PEMASOK (Kota;
```

Pada contoh ini, Kota pada relasi PEMASOK diindeks dan memungkinkan nilai pada Kota ada yang sama.

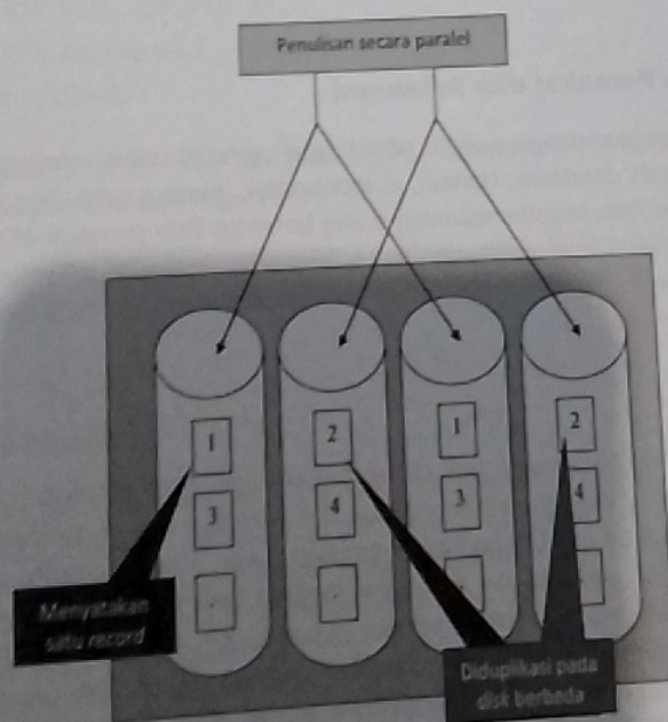
6.10 PENINGKATAN KINERJA AKSES DATA MELALUI RAID

Di luar teknologi DBMS, peningkatan kinerja akses terhadap data juga dapat dilakukan melalui penggunaan teknologi RAID (*Redundant Array of Independent Drives*). RAID menggunakan sejumlah *disk* dan menganggap *disk-disk* tersebut sebagai satu unit penyimpanan. Ketika DBMS menggunakan RAID sebagai tempat untuk menyimpan relasi, maka RAID akan melakukan *striping*. Yang dimaksud dengan *striping* adalah memecah data dan menyimpan pada *disk* yang terpisah. Gambar 6.21 memberikan ilustrasi tentang *striping*.

Keuntungan *striping* memungkinkan penulisan atau pengambilan data dapat dilakukan secara serentak pada sejumlah disk. Kemampuan memproses secara paralel inilah yang membuat kecepatan akses terhadap data meningkat. Selain itu, teknologi RAID juga memungkinkan penulisan secara otomatis untuk setiap data ke dalam beberapa disk. Teknik redundansi data ini dimaksudkan untuk menoleransi terhadap kegagalan disk (atau dikenal dengan istilah *fault tolerance*). Dengan cara seperti ini, apabila salah satu disk yang digunakan untuk menyimpan data mengalami masalah, data masih bisa diambil dari disk lain yang juga digunakan untuk menyimpan data. Gambarnya ditunjukkan pada Gambar 6.22.

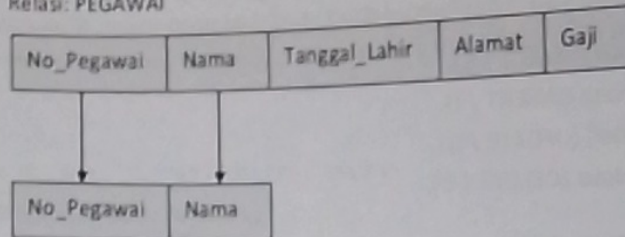
Catatan

Mirroring adalah teknik yang merekam secara otomatis suatu data ke lebih dari satu disk dengan tujuan kalau salah satu disk rusak data tetap tersedia pada disk lainnya.



Gambar 6.22 Gambaran *striping* dan *mirroring*.

Relasi: PEGAWAI



View: INFOPEG

Gambar 6.23 Gambaran view.

View dapat dibuat melalui perintah SQL bernama CREATE VIEW. Contoh:

```
CREATE VIEW INFOPEG AS  
SELECT No_Pegawai, Nama  
FROM PEGAWAI;
```

Perintah tersebut akan membuat view bernama INFOPEG berisi data No_Pegawai dan Nama yang berasal dari relasi PEGAWAI.

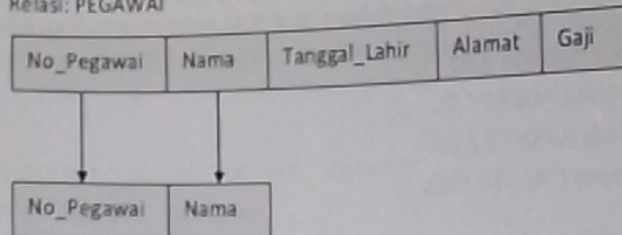
Suatu view juga bisa melibatkan sejumlah relasi. Dalam hal ini, bagian SELECT melibatkan kolom-kolom yang terdapat pada beberapa relasi.

6.11.3 Hak Akses

Terkait dengan hak akses, terdapat istilah wewenang atau *privilege*. Wewenang adalah tindakan-tindakan yang dapat dilakukan oleh seorang pemakai terhadap relasi atau view. Sebagai contoh, si A hanya boleh melihat isi relasi PEGAWAI tetapi si B boleh mengubah *record-record* yang terdapat pada relasi PEGAWAI. Pada contoh ini terlihat bahwa wewenang A dan B berbeda.

Tentu saja, wewenang sangat penting ditentukan dalam perancangan fisik karena secara fungsional setiap orang pun mempunyai hak yang berbeda-beda dalam mengakses

Relasi: PEGAWAI



View: INFOPEG

Gambar 6.23 Gambaran view.

View dapat dibuat melalui perintah SQL bernama CREATE VIEW. Contoh:

```
CREATE VIEW INFOPEG AS  
SELECT No_Pegawai, Nama  
FROM PEGAWAI;
```

Perintah tersebut akan membuat view bernama INFOPEG berisi data No_Pegawai dan Nama yang berasal dari relasi PEGAWAI.

Suatu view juga bisa melibatkan sejumlah relasi. Dalam hal ini, bagian SELECT melibatkan kolom-kolom yang terdapat pada beberapa relasi.

6.11.3 Hak Akses

Terkait dengan hak akses, terdapat istilah wewenang atau *privilege*. Wewenang adalah tindakan-tindakan yang dapat dilakukan oleh seorang pemakai terhadap relasi atau view. Sebagai contoh, si A hanya boleh melihat isi relasi PEGAWAI tetapi si B boleh mengubah *record-record* yang terdapat pada relasi PEGAWAI. Pada contoh ini terlihat bahwa wewenang A dan B berbeda.

Tentu saja, wewenang sangat penting ditentukan dalam perancangan fisik karena secara fungsional setiap orang pun mempunyai hak yang berbeda-beda dalam mengakses

3. *objek* menyatakan objek yang hak aksesnya akan diatur.
4. *pemakai* adalah nama pemakai yang telah didaftarkan pada sistem *database* dan sejumlah pemakai bisa disebutkan dengan antarpemakai dipisah dengan koma.

Contoh:

```
GRANT SELECT, INSERT, DELETE, UPDATE  
ON DBPEG.PEGAWAI TO adit;
```

Perintah ini merupakan pernyataan SQL pada MySQL digunakan untuk memberikan hak akses kepada pemakai *adit* untuk melakukan operasi *SELECT*, *INSERT*, *DELETE*, dan *UPDATE* terhadap relasi *PEGAWAI* yang terdapat pada *database* *DBPEG*.

DBMS seperti MySQL memungkinkan pemberian hak akses hanya terbatas pada kolom tertentu. Sebagai contoh, pemakai *terra* dapat melakukan *query* terhadap *Nomor_Pegawai* dan *Nama* saja, serta berhak mengubah isi *Nama*. Perintah yang diperlukan berupa:

```
GRANT SELECT (Nomor_Pegawai, Nama),  
UPDATE (Nama)  
ON DBPEG.PEGAWAI TO terra;
```

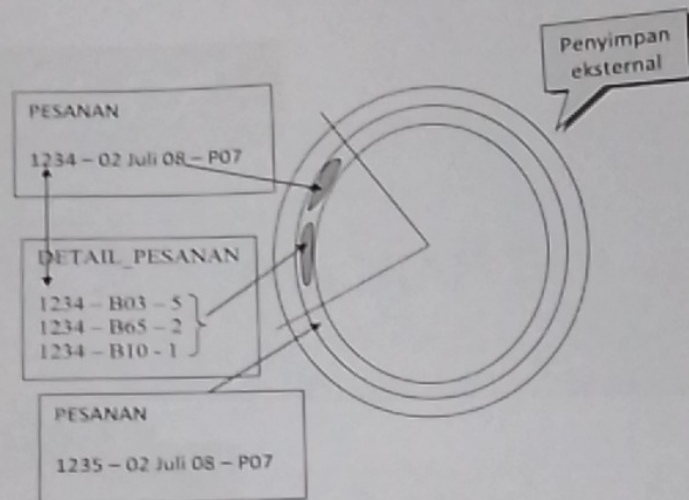
Kebalikan dari *GRANT*, tersedia perintah *REVOKE* yang berfungsi untuk mencabut sesuatu hak akses. Bentuk pernyataan ini:

```
REVOKE hak_akses ON objek  
FROM pemakai;
```

Contoh, pemakai dengan hak akses tertinggi yaitu *root* bisa memberikan perintah berikut:

```
REVOKE SELECT ON DBPEG.PEGAWAI.* FROM arif;
```

Dengan perintah seperti itu, pemakai *arif* tidak bisa lagi menggunakan *SELECT* terhadap relasi-relasi dalam *database* *DBPEG*.



Gambar 6.25 Ilustrasi *clustering* data.
Data yang berpasangan dalam relasi-relasi yang berbeda diletakkan bersebelahan.

Catatan

Oracle menyediakan perintah `CREATE CLUSTER` untuk membentuk *clustering* data.

Bab 7

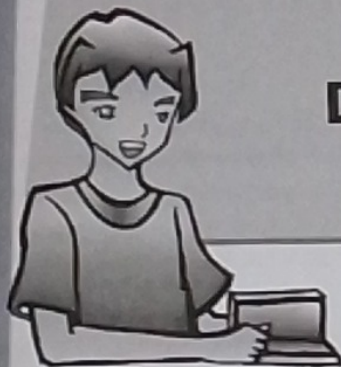


DIAGRAM EER

Bahasan:

- ✓ Pengenalan EER
- ✓ Penotasian dalam Hubungan Subtipe/Supertipe
- ✓ Pembentukan Hubungan Subtipe/Supertipe
- ✓ Penentuan Kekangan dalam Hubungan Subtipe/Supertipe
- ✓ Penyertaan Pembeda Subtipe
- ✓ Hierarki Tipe
- ✓ EER dan Hubungan ke Tipe Entitas
- ✓ Pemetaan EER ke Relasi